

Discrete Bayesian Optimization with Reparameterization for Edge-Cloud Configuration

Roberto Sala*, Bruno Guindani*, Alessandra Guglielmi[†] and Danilo Ardagna*

* *Department of Electronics, Information and Bioengineering, Politecnico di Milano, Milan, Italy*

[†] *Department of Mathematics, Politecnico di Milano, Milan, Italy*

Email: name.surname@polimi.it

Abstract—Bayesian Optimization (BO) is highly effective for solving complex optimization problems involving expensive black-box functions. However, existing methods often struggle with discrete, multidimensional, black-box constrained domains. These challenges are particularly relevant for tuning Edge-Cloud applications and High Performance Computing (HPC) systems, where meeting Quality of Service constraints is critical. We propose d-MALIBOO, a novel BO algorithm that leverages machine learning to estimate the feasible region and guide the search for the optimum. The method integrates a dynamic reparameterization strategy for the surrogate model parameters, avoiding repeated evaluations. We evaluate d-MALIBOO across a range of real-world scenarios, including an AI pipeline, big data queries, and computationally intensive HPC applications. In comparisons with state-of-the-art techniques, our algorithm achieves up to 142% lower regret and explores up to 39% more feasible configurations, while reducing wall-clock optimization time by a factor of 3–5. These findings show that d-MALIBOO provides a robust, efficient, and practical solution for constrained discrete optimization in real-world computing systems.

Index Terms—Bayesian Optimization, Discrete Constrained Optimization, Machine Learning, Computing Continuum, Cloud Computing, High-Performance Computing

I. INTRODUCTION

MODERN Information and Communication Technology (ICT) systems execute complex distributed applications across heterogeneous infrastructures. These applications often include Artificial Intelligence (AI) pipelines that require significant computational power and strict performance guarantees, with similar demands in Big Data analytics and scientific workloads typical of High-Performance Computing (HPC). As systems grow in scale, they become increasingly difficult to understand and configure due to the large number of interacting components. To meet computational and latency requirements, current distributed systems combine on-premise, edge, and cloud resources. In this context, the computing continuum has emerged as a key paradigm that integrates local edge servers with scalable cloud-based virtual machines (VMs), enabling efficient computation and privacy-aware data handling by reducing latency at the edge and providing elastic processing capacity in the cloud [1]–[3].

These applications must also satisfy strict Quality of Service (QoS) requirements such as response time or accuracy. These metrics are typically non-analytic and must be treated as black-box functions. For instance, optimizing a Neural Network (NN) application may involve selecting software parameters,

such as the number of hidden layers, and hardware resources, such as CPU or GPU memory, to maximize accuracy while meeting a target inference time.

Optimizing these applications requires solving constrained problems in discrete, high-dimensional domains. Optimization must be achieved with few executions since they are time-consuming and may involve multiple machines. Optimizing this complex infrastructure is challenging because certain applications perform better on specific hardware configurations. Consequently, simultaneous optimization of hardware and software parameters is crucial, as incorrect configurations can lead to significant costs for both providers and users [4].

Bayesian Optimization (BO) algorithms have emerged as a powerful tool for tackling optimization problems across a wide range of domains, including medicine, chemistry, information technology security, and AI [5]–[8], due to their efficiency in identifying near-optimal solutions within a limited number of iterations. The original BO algorithm [9] addresses global optimization problems involving complex black-box functions that are expensive to evaluate [10]. At the core of BO lies a probabilistic surrogate model, typically a Gaussian Process (GP), which approximates the objective function (OF) of the problem. This surrogate model serves two critical purposes: it estimates the value of the OF across the entire optimization domain and provides a measure of the uncertainty associated with these estimates. Using this surrogate model, the algorithm iteratively selects new points to evaluate by maximizing an acquisition function (AF). The AF quantifies the expected utility of evaluating the OF at a given point, balancing exploration of the domain with exploitation of promising regions.

The original BO algorithm was designed for problems involving continuous variables, unbounded domains, and continuous OFs [9], [10]. Several extensions have been proposed to adapt BO to discrete, mixed, and constrained domains [11]–[14]. Furthermore, BO algorithms are particularly well suited for optimizing edge-cloud applications and HPC systems [8], [15]–[17]. However, handling discrete variables and black-box constraints requires more sophisticated techniques to overcome the limitations of the original BO algorithm. The most challenging aspects include maximizing the AF in a discrete domain, modeling the relationship between inputs and black-box constraints, and avoiding repeated evaluations of previously observed points in the discrete space.

Our previous work [18] proposed MALIBOO, a BO algorithm designed to tackle constrained optimization problems.

This framework integrates Machine Learning (ML) into the BO framework to model the relationship between input configurations and black-box constraints.

This work introduces discrete MACHINE Learning In Bayesian OptimizatiOn (d-MALIBOO), a novel algorithm specifically designed to tackle discrete and black-box constrained optimization problems. First, the algorithm builds upon the original MALIBOO by implementing a new strategy for determining the feasible region within the discrete domain. Second, it incorporates various ML models into the search for the optimal solution, leveraging their strengths to approximate black-box functions. Additionally, the algorithm employs an ϵ -greedy approach to reduce the risk of stagnation at local optima, a challenge observed in the original work, and encourages exploration of the search space. Finally, we introduce a reparameterization strategy inspired by [11] to enhance the exploitation of promising regions within the discrete domain. This approach dynamically updates the parameters of the underlying surrogate model when attempting to visit a previously observed point. We further generalize our model by applying this technique to different GP kernels and AFs than [11].

The main contributions of this work are as follows:

- 1) We extend the reparameterization method proposed in [11] to different GP kernels and AFs.
- 2) We evaluate the performance of d-MALIBOO across multiple real-world scenarios with diverse constraints, identifying the best-performing configuration based on regret, feasible exploration rate, and total execution time. Tested applications include an AI pipeline, big data queries, and computationally intensive HPC workloads.
- 3) We compare d-MALIBOO against several alternative approaches, demonstrating that it consistently achieves superior performance across all considered scenarios. For instance, d-MALIBOO attains 142% lower average regret than OpenTuner [19], explores 39% more feasible configurations than SVM-CBO [12], and achieves 3 to 5 times lower wall-clock optimization time than BO-PR [13] while maintaining comparable regret.

This paper is organized as follows. Section II reviews the related work. Section III outlines the optimization problem we address. Section IV provides an overview of BO. Section V details the components of d-MALIBOO, in particular the reparameterization strategy. Section VI introduces the setup of the experimental campaign, while Section VII presents the numerical evaluation and discusses the proposed algorithm in light of the results. Finally, Section VIII concludes the paper and outlines directions for future research.

II. RELATED WORK

BO is a versatile algorithm capable of addressing a wide range of optimization problems across different domains. In this section, we first review works on the optimization of black-box functions in the context of ICT systems (Section II-A). Next, we focus on BO-based techniques and examine approaches for constrained and discrete domains (Section II-B). Finally, we describe the approaches used for comparison with d-MALIBOO (Section II-C).

A. Black-box functions optimization

The optimization of black-box functions has become a prominent topic in recent research, particularly with the rapid growth of ML and AI. BO is especially well suited to this setting, where only a few iterations are available to find a near-optimal solution. Two seminal works in this field are worth highlighting. Hyperopt [20] introduces an open-source framework for hyperparameter optimization, including deep learning models. Google Vizier [21] presents a state-of-the-art system for black-box optimization that supports multiple algorithms and is designed for large-scale applications.

These findings have promoted the use of BO in multiple contexts, ranging from hyperparameter optimization to system-level performance tuning. For example, [6] proposes an intrusion detection framework based on ML classifiers, such as Support Vector Machines (SVM), Random Forests, and k-Nearest Neighbors, and optimized with BO. In HPC, [22] applies BO to automate benchmark parameter tuning on supercomputers, improving energy efficiency and enabling their system, Kukai, to achieve second place in the Green500 ranking. Furthermore, [23] introduces an active learning framework that uses BO to identify application and platform parameters leading to high-performing configurations. In neural architecture search, [7] combines BO with optimal transport to optimize Neural Network (NN) architectures. Edge computing applications are explored by [8], which proposes a multimodal BO framework for many-to-many offloading of AI workloads on IoT devices, and by [15], which presents a BO method for task offloading and resource allocation in time-varying multi-server, multi-user mobile edge computing systems. Finally, [24] studies the performance and robustness of BO algorithms across problems of increasing complexity and demonstrates their applicability to computational fluid dynamics problems.

More recent work explores generative model-based approaches for optimization. For example, [25] proposes the Denoising Diffusion Optimization Models approach, which leverages conditional diffusion models to learn the inverse mapping from function values to inputs. Similarly, [26] investigates the potential of LLMs for black-box optimization, highlighting their ability to process diverse input data, provide scalability guarantees, and generalize effectively through in-context learning. Also, [27] explores the use of diffusion models to model complex distributions for black-box optimization.

Unlike these works, we propose a framework that can be used either as a lightweight optimization component within a larger system or as the primary algorithm for a discrete optimization problem. Furthermore, d-MALIBOO supports optimization under black-box constraints.

This paper extends our previous work [28], which focused on the machine learning techniques implemented in the BO algorithm. Here, we extend that work methodologically by: (i) introducing novel reparameterization techniques, (ii) expanding the experimental campaign, and (iii) comparing our approach with several alternative literature methods as presented in Section II-C.

TABLE I: Comparison between d-MALIBOO and competitor approaches.

Algorithm	BO technique	Model on OF	Constraints handling	Repetition avoidance technique	Reference
MALIBOO	✓	GP	ML model	Memory queue	[18]
Discrete-BO	✓	GP	✗	Reparametrization of GP and AF	[11]
SVM-CBO	✓	GP	SVM model	✗	[12]
BO-PR	✓	GP and Probabilistic model	Probabilistic model	✗	[13]
OpenTuner	✗	Ensemble methods	Conditional Objective	✗	[19]
d-MALIBOO	✓	GP and ML model	ML model	Reparametrization of GP and AF	this work

B. BO in constrained and discrete domains

Extending BO from continuous to discrete and mixed-variable domains has become an active research area, driven by challenges in combinatorial spaces, categorical variables, and integer constraints. Several studies have proposed techniques for modeling, constraint handling, and efficient exploration in these complex domains. For example, [16] presents a probabilistic algorithm to handle noisy constraints and allows decoupled evaluations of objective and constraint functions. Building on this approach, [17] proposes predictive entropy search with constraints, which decomposes the information gain into contributions from each independent function, improving optimization efficiency.

Another major area of development is BO for combinatorial and categorical variables. [29] presents BO of combinatorial structures, employing convex optimization techniques to efficiently approximate the maximum of the AF. Similarly, [30] proposes an approach for mixed-variable optimization by merging multi-armed bandits with BO. Focusing on optimization in discrete domains, [14] introduces a novel covariance function that captures the fact that the OF remains constant in intervals rounded to the same integer, addressing limitations of common approaches such as rounding integer variables or using one-hot encoding for categorical variables.

Finally, some recent works focus on the computational efficiency and scalability of BO. In particular, [31] presents the Scalable Constrained Bayesian Optimization algorithm, which employs trust regions, batched observations, and an extended Thompson sampling approach for high-dimensional problems. [32] introduces a continuous relaxation of the OF along with a covariance function whose runtime scales linearly with input length, facilitating efficient inference and optimization in mixed-variable settings.

Unlike previous works, d-MALIBOO leverages the knowledge of observed points to simultaneously estimate the feasible region based on evaluations of the black-box constraints through ML models and search for the optimum within this region. It is tailored for discrete domains, making it suitable for a variety of cases that include categorical and combinatorial variables. Finally, its computational efficiency (that we will demonstrate with our experimental campaign) ensures scalability with respect to the domain dimension.

C. Comparison approaches

We now introduce the algorithms from the literature that we compare with d-MALIBOO in our experimental evaluation. Table I summarizes the key differences among all approaches.

1) *Foundational works*: Our novel approach builds upon two key previous works: MALIBOO [18] and Discrete-BO [11]. MALIBOO is a BO-based algorithm that leverages ML models to estimate the feasible region under black-box constraints. Building on this framework, we propose a method specifically designed for discrete variables, extending the use of ML models beyond constraint modeling, also to guide the search for the optimum, and we introduce a strategy to avoid resampling of previously evaluated points. Discrete-BO is another BO-based algorithm that, for the first time, introduces a reparameterization strategy for both the GP kernel and the AF to prevent redundant evaluations. However, this framework is limited to unconstrained optimization. In this work, we extend the proposed strategy to support multiple GP kernels and AFs, integrating it into our framework for discrete variables and black-box constraints.

2) *Support Vector Machine Constrained BO (SVM-CBO)*: This algorithm leverages SVM to identify the feasible region from black-box constraints and then applies BO within this region to find the optimum [12]. The method operates in two phases. In the first phase, it selects points for evaluation by balancing two objectives: the distance from the current boundaries, which refines the region estimate, and a measure of search space coverage, which helps identify potentially disconnected feasible regions. In the second phase, it applies a modified BO algorithm with the following changes: (i) the search space is constrained to the region identified in the first phase rather than the full domain, and (ii) a GP is fit to construct a probabilistic surrogate model of the OF using only the feasible points observed so far. At each iteration, both the SVM and GP models are updated.

Unlike SVM-CBO, our d-MALIBOO algorithm also leverages information from unfeasible points to evaluate both the objective and constraint functions. This detail is crucial because the optimal solution often lies near the boundary of the feasible region, and the proportion of feasible points in a discrete domain may be scarce in certain cases.

3) *BO with Probabilistic Reparameterization (BO-PR)*: This approach enhances the optimization of the AF by introducing a probabilistic reparameterization of discrete variables [13]. Specifically, (1) is relaxed by defining a discrete probability distribution $p(Z, \vartheta)$ over the discrete variables Z , with support over the original domain $\mathbb{Z} \subset \mathbb{R}^d$. This distribution is parameterized by continuous parameters ϑ_i , resulting in a vector $\vartheta = \vartheta_1, \dots, \vartheta_d$ that maps the entire discrete domain. The authors then define a probabilistic objective (PO) as the expectation of the AF under this distribution: $\mathbb{E}_{Z \sim p(Z|\vartheta)}[AF(Z)]$. The PO serves as a new AF, whose maximizer determines the next point to evaluate. In this framework, the original AF is optimized indirectly via the continuous parameters ϑ rather

than directly over the discrete domain. The maximizer of the PO coincides with that of the original AF.

Like our approach, BO-PR can handle black-box constraints. However, aside from the probabilistic reparameterization of discrete variables, it does not explicitly address the risk of evaluating previously observed points.

4) *OpenTuner*: OpenTuner is a widely used autotuner that employs an ensemble of search techniques to collaboratively explore large, complex search spaces [19], with each technique responsible for testing and identifying optimal candidate configurations. It maintains a shared database of configurations, allowing ensemble members to incorporate information from others into their search process. Techniques that demonstrate superior performance by finding better configurations are allocated more resources, with OpenTuner managing this allocation by solving a multi-armed bandit problem using the area-under-the-curve credit assignment metric.

This approach differs significantly from d-MALIBOO, as it employs an ensemble of non-BO methods. We choose it as a competitor because it is a popular framework for constrained optimization, particularly in the HPC community [33].

III. PROBLEM FORMULATION

We focus on edge-cloud and HPC systems, where tuning both hardware and software parameters is crucial for minimizing a specific OF, while adhering to performance constraints. The general optimization problem is formalized as follows. Let $D \subset \mathbb{R}^d$ be a d -dimensional discrete domain containing all possible system configurations, and let $\mathbf{x} \in D$ denote a vector of parameters. We do not require D to be a complete grid, nor must the values in each dimension be equispaced, an assumption sometimes used in the literature [14].

Let $f(\cdot)$ denote the metric to minimize. This quantity is a *black-box function* as its analytical formulation is unknown, along with its mathematical properties such as concavity. Furthermore, evaluating f is often expensive in terms of computational resources or cost. For example, f may represent the cost of running an application on an edge-cloud infrastructure, a performance metric such as the error on a cross-validation set of a training algorithm, or the score of a molecular docking simulation. The input vector $\mathbf{x}$ includes parameters such as the number of nodes, virtual machines (VMs), CPU cores, memory allocation, workload size, number of iterations, and other relevant quantities.

In addition, let $g_j(\cdot) \in [G_{min}^j, G_{max}^j]$, $j = 1, \dots, C$, denote C scalar, black-box constraint functions. These functions are not necessarily independent of f , as this assumption does not always hold in practice. The constraint functions may represent application-specific performance requirements or quality-of-experience constraints, such as minimum accuracy or maximum response time. Due to their black-box nature, information about f and g_j can only be obtained through direct evaluation.

Our problem formulation follows:

$$\begin{aligned} & \min_{\mathbf{x} \in D} f(\mathbf{x}) + \eta \\ & \text{s.t. } g_j(\mathbf{x}) \in [G_{min}^j, G_{max}^j], \quad j = 1, \dots, C \end{aligned} \quad (1)$$

Here, η represents random noise in the OF evaluation, arising from factors such as resource contention or network congestion, which introduce variability. Typically, this noise is modeled using a Gaussian distribution [22].

This general formulation is broadly applicable to a variety of scenarios ranging from HPC configuration to ML hyperparameter tuning and computing continuum applications.

IV. BAYESIAN OPTIMIZATION BACKGROUND

BO is a highly efficient method for optimizing black-box OFs, as it can identify near-optimal solutions within a few evaluations. This efficiency is particularly important when each evaluation of the OF incurs significant computational cost or requires a considerable amount of time to execute. The problem becomes especially critical in discrete domains, where the number of possible values for each variable can lead to a combinatorial explosion in the total number of potential configurations. Consider, for example, tuning AI models that require a large number of VMs, or executing a drug discovery application in which each evaluation involves lengthy computations or extensive laboratory experiments. In these cases, limiting the number of configurations explored is crucial for effectively controlling costs.

The core idea behind BO is to treat each value of $f(\cdot)$ as a random variable, with a *prior* distribution capturing the initial knowledge about the OF. This knowledge is iteratively refined with each new evaluation. We now present the key elements of the BO algorithm, including the model used to approximate the OF, typically a GP, and the strategy for selecting the next evaluation, namely the AF.

A *Gaussian Process* (GP) is a statistical model that assumes each unknown value of $f(\mathbf{x})$ is drawn from a Gaussian distribution with mean $\mu_n(\mathbf{x})$ and variance $\sigma_n^2(\mathbf{x})$, both of which depend on $\mathbf{x}$ and the n previously observed points. Specifically, given a history $H_n = \{(\mathbf{x}_i, f(\mathbf{x}_i)) \mid i = 1, \dots, n\}$ and a point $\mathbf{x} \in D$, the distribution of $f(\mathbf{x})$ is:

$$f(\mathbf{x})|H_n \sim \pi_{\mathbf{x}}(\cdot|H_n) = N(\mu_n(\mathbf{x}), \sigma_n^2(\mathbf{x}, \mathbf{x})). \quad (2)$$

The functions $\mu_0(\cdot)$ and $\sigma_0^2(\cdot, \cdot)$, known as the mean and kernel functions, respectively, serve as hyperparameters of the GP model and define the *prior distribution* of the OF. These functions are updated after each evaluation of the OF, resulting in a *posterior distribution* characterized by $\mu_n(\cdot)$ and $\sigma_n^2(\cdot)$, as indicated in (2). While a constant mean function $\mu_0(\cdot) = \mu_0$ is commonly assumed, the choice of the kernel varies considerably across the literature. The following are the most popular options [10], [34]:

- *Radial Basis Function* (RBF) kernel:

$$\sigma_0^2(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{d(\mathbf{x}_i, \mathbf{x}_j)^2}{2l^2}\right), \quad (3)$$

where $d(\cdot, \cdot)$ is the Euclidean distance and $l > 0$ is a length-scale parameter.

- *Matérn kernel*:

$$\sigma_0^2(\mathbf{x}_i, \mathbf{x}_j) = \frac{2^{1-\nu} (\sqrt{2\nu} d(\mathbf{x}_i, \mathbf{x}_j))^\nu K_\nu(\sqrt{2\nu} d(\mathbf{x}_i, \mathbf{x}_j))}{\Gamma(\nu)}, \quad (4)$$

where $\nu > 0$ is the smoothness parameter, $\Gamma(\cdot)$ and $K_\nu(\cdot)$ are the gamma and modified Bessel functions.

In this work, we consider both kernels using various reparameterization strategies. In particular, the two parameters l and ν introduced here are essential elements of our adaptive method.

The *Acquisition Function* (AF) $\alpha(\mathbf{x}|H_n)$, or simply $\alpha(\mathbf{x})$, leverages the current knowledge of the OF to determine which point in the domain should be evaluated in the next iteration. Specifically, it assesses the expected utility of evaluating $f(\mathbf{x})$ at a given point in the domain D , assigning higher values to regions that are either under-explored (i.e., regions with few evaluations) or appear promising based on previous results. This mechanism effectively balances exploration and exploitation, allowing the BO algorithm to initially explore the domain and subsequently focus on refining the search for the global optimum as more information is gathered. The AF depends on both the current history H_n and the model underlying the OF. However, it can be efficiently evaluated in closed form and is maximized at each iteration of the BO algorithm using fast heuristics such as gradient-based methods. We focus on two established AFs from the literature [10], [18]:

- *Expected Improvement* (EI):

$$EI_n(\mathbf{x}) := \mathbb{E}_{\pi_{\mathbf{x}}(\cdot|H_n)}[\max(f_n^* - f(\mathbf{x}), 0)], \quad (5)$$

where $f_n^* = \min_{i \leq n} f(\mathbf{x}_i)$ denotes the minimum value observed up to the current iteration, and $\pi_{\mathbf{x}}(\cdot|H_n)$ represents the current posterior distribution of $f(\mathbf{x})$. This AF maximizes the expected improvement over the current minimum, given the information from the history H_n .

- *Upper Confidence Bound* (UCB):

$$UCB_n(\mathbf{x}) = -\mu_n(\mathbf{x}) + \lambda\sigma_n(\mathbf{x}), \quad (6)$$

where λ balances the uncertainty in the predictions of the GP model against the mean function. Larger values of λ promote exploration of under-explored regions with higher uncertainty, while smaller values favor exploitation by prioritizing points with better expected performance (i.e., lower values of the OF). Note that the negative sign is introduced to account for the minimization problem in Equation (1), whereas the AF itself is always maximized.

The classical BO algorithm does not account for black-box constraints. MALIBOO [18] addresses this limitation by leveraging an ML model to estimate the feasible region and by incorporating these predictions directly into the AF. Moreover, BO was originally developed for continuous optimization problems [10]. In the following section, we introduce our d-MALIBOO algorithm, specifically designed to solve constrained optimization problems in discrete domains.

V. D-MALIBOO

d-MALIBOO is an iterative BO-based algorithm designed to optimize edge-cloud and HPC application configurations under a discrete optimization domain and black-box constraints. d-MALIBOO introduces several strategies to enhance both the exploration and exploitation capabilities of our algorithm by leveraging ML models, particularly in discrete domains. We also address another major challenge in discrete BO, namely

the repeated evaluation of already sampled points. Specifically, as uncertainty decreases over the iterations, the OF's surrogate model may fail to escape local minima or to explore the regions surrounding high-performing points that have already been evaluated. This work extends the reparameterization technique introduced by [11] to tackle this issue.

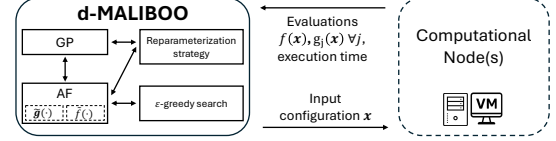


Fig. 1: d-MALIBOO Architecture.

Figure 1 illustrates the architecture of the framework implementing d-MALIBOO. At each iteration, the AF module leverages the ML models for the constraints and the OF, together with information from the GP, to select the next configuration $\mathbf{x}$ to evaluate. d-MALIBOO then sends $\mathbf{x}$ to a computational node, which executes the application under that configuration and returns the evaluations of the OF $f(\mathbf{x})$ and constraint functions $g_j(\mathbf{x})$. The reparameterization module interacts with both the GP and the AF when hyperparameter updates are triggered, while the ε -greedy module leverages the ML models within the AF module to support exploration.

Section V-A describes the main components of d-MALIBOO and the challenges they address. Section V-B illustrates the complete optimization algorithm, and Section V-C discusses convergence guarantees.

A. d-MALIBOO modules

The modules of the d-MALIBOO framework implement: 1) a GP regression engine, 2) a novel AF integrated with ML-based approaches to estimate the feasible region, 3) an ε -greedy search strategy to encourage exploration, and 4) a reparameterization strategy for the GP and AF that triggers when the AF suggests an already evaluated point.

1) *Gaussian Process module*: The GP is the core component of classical BO algorithms and serves as the surrogate model estimating the OF, as described in Section IV. At each algorithm iteration, the GP module receives the point selected by the AF along with its OF evaluation on the computational node. It appends this information to the history H_n of visited points and then performs a GP posterior update following (2).

2) *Acquisition function module*: This module computes the next point to evaluate in the BO algorithm by maximizing a novel AF. In particular, we introduce a novel probabilistic approach for the estimation of the feasible region leveraging ML models and their ability to infer black-box input-output relationships. Let $D' = \{\mathbf{x} \in D \mid g_j(\mathbf{x}) \in [G_{min}^j, G_{max}^j] \forall j = 1, \dots, C\}$ be the feasible region of the search space. At each algorithm iteration, one ML model $\tilde{g}(\cdot)$ is trained for all constraints $\mathbf{g}(\cdot) = \{g_j(\cdot)\}_{j=1, \dots, C}$ to infer whether $\mathbf{x} \in D'$. The training dataset for $\tilde{g}(\cdot)$ consists of the collection of previously visited points as covariates and the corresponding values of $\mathbf{g}(\cdot)$ as the target. Given a point $\mathbf{x}$, this model acts as a classifier, providing the probability that $\mathbf{g}(\mathbf{x}) \in D'$. We then define the AF for our BO algorithm as follows:

$$\bar{\alpha}(\mathbf{x}) = \alpha(\mathbf{x}) \mathbb{P}(\tilde{g}(\mathbf{x}) \in D') \quad (7)$$

where $\alpha(\cdot)$ is the original AF from classical BO (such as EI and UCB). We refer to this as the *probability* approach. This method applies a smooth penalization of the AF near the boundary of the feasible region, reducing the risk of excluding optimal points, particularly during early iterations when the ML models may still be inaccurate.

This idea is inspired by MALIBOO [18], which adopts a more naïve strategy based on binary indicator functions rather than probabilities:

$$\bar{\alpha}(\mathbf{x}) = \alpha(\mathbf{x}) \mathbb{1}\{\tilde{g}(\mathbf{x}) \in D'\}. \quad (8)$$

We refer to this as the *indicator* approach, of which the probability approach is a natural generalization. While the indicator strategy is expected to be more effective for simpler constraint functions, the probability approach tends to perform better when the constraints are more complex or nonlinear. However, the black-box nature of the problem makes it difficult to predict which method will be superior in a given setting. For this reason, we implement both approaches within the d-MALIBOO framework.

Similarly to the estimation of the feasible region via ML, we also introduce two approaches that incorporate ML predictions for the OF. Specifically, we train an additional ML model $\tilde{f}(\cdot)$ using the history of visited points as covariates and the corresponding OF values as the target. The *indicator* approach predicts whether the OF evaluated at $\mathbf{x}$ yields a lower value than the current minimum f_n^* , resulting in the following AF:

$$\bar{\alpha}(\mathbf{x}) = \alpha(\mathbf{x}) \mathbb{1}\{\tilde{f}(\mathbf{x}) \leq f_n^*\}. \quad (9)$$

On the other hand, the *probability* approach evaluates the probability that the OF at $\mathbf{x}$ yields a lower value than f_n^* , resulting in a smoother penalization of the AF:

$$\bar{\alpha}(\mathbf{x}) = \alpha(\mathbf{x}) \mathbb{P}(\tilde{f}(\mathbf{x}) \leq f_n^*). \quad (10)$$

As with the estimation of the feasible region, the latter approach mitigates the risk of prematurely discarding potentially optimal points. However, the relative performance of the two approaches depends on the specific optimization problem.

The ML models for the feasible region and the OF are implemented independently as two separate submodules. They can be used individually or together. In the latter case, their respective estimates are combined by multiplying them in the definition of the modified AF.

3) *ε -greedy search module*: Balancing exploration and exploitation is essential to prevent stagnation during optimization. To this end, we implement an ε -greedy strategy, widely adopted in reinforcement learning literature [35], to reduce the risk of premature convergence to local minima. At each iteration, with probability ε , this module randomly selects the next point to evaluate from the current estimated feasible domain, instead of maximizing the AF as usual.

4) *Reparameterization module*: The fourth module implements the d-MALIBOO reparameterization strategies.

Optimizing a function on discrete domains carries the risk of evaluating the same points multiple times, especially when the algorithm has sufficiently explored the domain and tends to focus on a specific region of the search space. Naïve

approaches proposed in the literature, such as forcing the AF to vanish at recently visited points [18], may introduce undesirable side effects, including prematurely abandoning exploitation in promising regions.

With these motivations, we address this issue by dynamically updating the hyperparameters of the underlying surrogate model when the AF suggests a previously visited point, extending ideas originally introduced in Discrete-BO [11]. When reparameterization is triggered, the module computes a new set of hyperparameters and forwards them to the GP and AF modules to update their respective models. Changes in the model hyperparameters alter the shape of the AF, leading to a different maximizer selected for evaluation.

We now describe our reparameterization strategies. Consider an introductory example taken from [11], in which the surrogate GP model employs an RBF kernel together with the UCB AF. The RBF kernel is parameterized by the length scale l (see Eq. (3)), while the UCB AF is characterized by the hyperparameter λ . This hyperparameter governs the exploration-exploitation trade-off of the algorithm and provides a clear interpretation of its effect. Specifically, low values of λ favor exploitation, as the AF is dominated by the mean function of the GP kernel (see Eq. (6)). Conversely, large values of λ cause uncertainty to prevail over the mean, thereby promoting exploration of the domain. However, once the BO algorithm begins re-evaluating points it has previously observed, the mean is already dominating over the variance. Our reparameterization strategy proposes increasing the value of the trade-off hyperparameter λ to help the algorithm escape local optima, and adjusting the length scale l so that the new suggested point $\mathbf{x}'_{n+1}$ is still close to the original point. Specifically, we compute the new λ_n^* and l_n^* values by solving the following auxiliary optimization problem:

$$\lambda_n^*, l_n^* = \arg \min_{\lambda \in [\lambda_n, \lambda_n + \lambda_h], l \in (0, l_h]} g(\lambda, l), \quad (11)$$

$$g(\lambda, l) = (\lambda - \lambda_n) + d(\mathbf{x}_{n+1}, \mathbf{x}'_{n+1}) + M \mathbb{I}\{\mathbf{x}'_{n+1} \in H_n\}.$$

Here, n denotes the current BO iteration, while $\mathbf{x}'_{n+1}$ represents the new point that maximizes the AF after reparameterization. λ_n is the current trade-off hyperparameter, and λ_h is the maximum variation of λ from its current value, introduced to prevent pure exploration of the domain. l_h denotes the maximum length scale, beyond which the model becomes overly smooth and tends to under-explore due to excessive correlation between distant points. M is a sufficiently large constant that penalizes the already observed points.

Problem (11) consists of three terms. The first ensures that λ_n^* does not significantly exceed the original λ_n , preventing the algorithm from becoming overly explorative. The second minimizes the distance between the previously suggested point and the new one, encouraging the algorithm to stay in a neighborhood of the original point. The third penalizes already observed points to avoid re-evaluating previous observations. After reparameterization, the algorithm proceeds by selecting the next points according to the updated surrogate model until the next reparameterization occurs.

The authors of [11] do not evaluate the effectiveness of this approach with different GP kernels or AFs, nor do they consider black-box constraints. As part of our contribution, we show how to extend the reparameterization technique to all GP kernels and AFs introduced in Section IV, as well as their possible combinations.

Consider the Matérn kernel, which is widely adopted in many practical applications [36]. It depends on the smoothness hyperparameter $\nu \geq 1/2$ (see Eq. (4)), with the most common values being $5/2$ and $3/2$ [34]. When employing the Matérn kernel together with the UCB AF, we obtain the new hyperparameters λ_n^* and ν_n^* by solving the following problem:

$$\lambda_n^*, \nu_n^* = \arg \min_{\lambda \in [\lambda_n, \lambda_n + \lambda_h], \nu \in (\frac{1}{2}, \nu_h]} g(\lambda, \nu), \quad (12)$$

$$g(\lambda, \nu) = (\lambda - \lambda_n) + d(\mathbf{x}_{n+1}, \mathbf{x}'_{n+1}) + M \mathbb{I}\{\mathbf{x}'_{n+1} \in H_n\}.$$

In this case, ν_h represents the maximum value allowed for the smoothness hyperparameter. An excessively large value may lead to an over-smoothed surrogate model, potentially leading to under-exploration of certain regions and underestimation of uncertainty. As in the previous case, the goal of this reparameterization is to identify a combination of the hyperparameters λ and ν that ensures the AF suggests a new, previously unobserved point $\mathbf{x}'_{n+1}$, as close as possible to the previously suggested point $\mathbf{x}_{n+1}$, while minimizing variations in the exploration hyperparameter λ .

Similarly, we extend the reparameterization technique to the EI AF. Note that, unlike UCB, EI has no explicit hyperparameters but implicitly depends on the GP kernel, as it represents an expectation with respect to the GP posterior distribution (see Eq.(5)). When employing the RBF kernel together with the EI AF, we perform reparameterization by solving:

$$l_n^* = \arg \min_{l \in (0, l_h]} g(l), \quad (13)$$

$$g(l) = d(\mathbf{x}_{n+1}, \mathbf{x}'_{n+1}) + M \mathbb{I}\{\mathbf{x}'_{n+1} \in H_n\}.$$

If we use the Matérn kernel with the EI AF, we obtain:

$$\nu_n^* = \arg \min_{\nu \in (\frac{1}{2}, \nu_h]} g(\nu), \quad (14)$$

$$g(\nu) = d(\mathbf{x}_{n+1}, \mathbf{x}'_{n+1}) + M \mathbb{I}\{\mathbf{x}'_{n+1} \in H_n\}.$$

Note that this approach can be extended to other AFs, such as the Probability of Improvement and the Knowledge Gradient [10], but this work focuses on the most widely used AFs.

B. The algorithm

We summarize our proposed approach in Algorithm 1. The algorithm takes as input the number of initial points n_0 and BO iterations N ; the GP, including its mean and covariance functions; the ML model approaches for constraints (Eqs. (8)-(7)) and for the OF (Eqs. (9)-(10)); and the probability ε used in the ε -greedy strategy.

d-MALIBOO requires the number n_0 of an initial set of points to construct the prior distribution of the surrogate

model. To this end, we employ Latin Hypercube Sampling (LHS) [37], which promotes well-distributed coverage of the domain from the first evaluations. LHS stratifies each parameter range into equally probable intervals and draws one random sample from each, increasing point diversity and improving the accuracy of the surrogate model in the early iterations. After sampling the initial points and evaluating both the objective and constraint functions at these points (lines 2-3), the BO iterations begin. At each iteration, the ML models used to estimate the constraint functions ($\tilde{g}_j(\cdot) \forall j$) are updated using the current history H_n (line 5). The ε -greedy algorithm is triggered by sampling from a Bernoulli distribution with parameter ε . With probability ε , the next point is selected randomly within the feasible domain estimated by the ML models (lines 6-8). Otherwise, the algorithm updates the ML model for the OF, the GP surrogate model, and the AF using the most recent data, and maximizes the latter to suggest the next point to be evaluated (lines 10-12). If the suggested point has already been observed (i.e., $\mathbf{x}_{n+1} \in H_n$), the reparameterization strategy is applied by solving one of the auxiliary problems (11)–(14). A new point that has not yet been observed is then evaluated (lines 13-16). At the end of the N iterations, the algorithm returns the point that minimizes the OF among those in the history, identified within the discrete domain and that satisfies the black-box constraints.

From a computational complexity standpoint, the most demanding operation is the update of the GP kernel, as it involves inverting an $n \times n$ matrix, which has a complexity of $O(n^3)$, where n is the number of points in the history H_n , which implies $O(N^4)$ worst case complexity. However, in the BO framework, it is generally assumed that the computational cost of evaluating the OF dominates the update cost of the surrogate model. We will examine this assumption and compare the performance of our approach with respect to state-of-the-art methods in the experimental campaign in Section VII.

C. Convergence analysis

We briefly discuss the convergence guarantees of our d-MALIBOO algorithm. The authors of [11, §4] prove that when reparameterization is triggered, i.e., when (11) is solved, during both the exploration and exploitation phases of their algorithm, there always exists a surrogate model that proposes an unseen point in the discrete domain (as long as the entire domain has not been visited yet). Although the proof was originally shown using problem (11), it can be extended to problems (12)–(14) by considering the following two aspects. First, consider that d-MALIBOO always selects new points by maximizing the AF under the current surrogate model (except when the ε -greedy exploration is triggered, but this case does not influence the analysis). To guarantee the AF to select a different point than $\mathbf{x}_{n+1}$, the hyperparameters of the surrogate model must change. Second, if the penalization M for repeated points is greater than the diameter of the optimization domain, the last term in (11), (13), (12) and (14) dominates the other terms. As a consequence, the auxiliary problems yield a different set of hyperparameters for the surrogate model, which leads the AF to suggest an unobserved point in the discrete

Algorithm 1 *d-MALIBOO algorithm*

```

1: Input:  $n_0, N, GP, ML\ constr., ML\ target, \varepsilon$ 
2: Pick  $n_0$  initial points through LHS
3:  $H_n \leftarrow \{(\mathbf{x}_i, f(\mathbf{x}_i)), i = 1, \dots, n_0\}$ 
4: for iterations  $n = 1 : N$  do
5:   Train ML constr. model with data in  $H_n$ 
6:    $g \leftarrow$  sample from Bernoulli( $\varepsilon$ )
7:   if  $g == 1$  then
8:      $\mathbf{x}_{n+1} \leftarrow$  random point within ML constr.
9:   else
10:    Train ML target model with data in  $H_n$ 
11:    Update GP model and  $\tilde{\alpha}(\cdot)$  with data in  $H_n$ 
12:     $\mathbf{x}_{n+1} \leftarrow \arg \max \tilde{\alpha}(\cdot)$ 
13:    if  $\mathbf{x}_{n+1} \in H_n$  then
14:      Reparametrize GP kernel and AF by one of
        Eqs. (11)–(14)
15:       $\mathbf{x}_{n+1} \leftarrow \arg \max \tilde{\alpha}(\cdot)$ 
16:    end if
17:  end if
18:  Evaluate  $f(\mathbf{x}_{n+1})$ 
19:   $H_n \leftarrow H_n \cup \{(\mathbf{x}_{n+1}, f(\mathbf{x}_{n+1}))\}$ 
20: end for
21: return  $\hat{\mathbf{x}} = \arg \min_{g_j(\mathbf{x}) \in [G_{min}^j, G_{max}^j] \forall j, \mathbf{x} \in H_n} f(\mathbf{x})$ 

```

domain. As the number of iterations increases, the algorithm always visits points within the discrete domain that have not been evaluated before, eventually leading to convergence.

VI. DESIGN OF THE EVALUATION

This section presents the setup of our experimental campaign. Section VI-A describes the test function and applications under study, Section VI-B details the configuration of d-MALIBOO and the SOTA approaches used for comparison, and Section VI-C introduces the evaluation metrics.

A. Optimization scenarios

The optimization scenarios in which we evaluate d-MALIBOO employ the Schwefel function and four real-world applications, as described below.

The *Schwefel* function is a well-known benchmark in the unconstrained optimization literature [38], [39]. We use it as the OF in an ablation study of the individual components of our d-MALIBOO algorithm and in a scalability analysis (Section VII-A). It is defined as follows:

$$f(\mathbf{x}) = 418.9829 \cdot d - \sum_{i=1}^d x_i \cdot \sin\left(\sqrt{|x_i|}\right). \quad (15)$$

We choose the following constraint:

$$g(\mathbf{x}) = \max\left(\sqrt{|x_i|}\right) \in [0, G_{max}]. \quad (16)$$

For the ablation study, we fix $d = 8$ and set $G_{max} = 20.41$, resulting in a feasible domain that is 20% the size of the entire search space. The discrete domain consists of 200^d random points uniformly sampled from the interval $[-500.0, 500.0]^d$.

In the subsequent scalability analysis, we vary the dimension of the discrete domain as $d = \{4, 8, 12, 16\}$. To maintain the feasible domain at 20%, we adjust the upper bound G_{max} , which also shifts the location of the optimum. The 20% figure serves as a baseline for feasible exploration rates, as it corresponds to the expected portion of feasible points when selecting randomly. For all experiments with the Schwefel function, we sample $n_0 = 10$ initial points and perform $N = 200$ algorithm iterations. Note that the total number of points in the discrete domain (200^d) makes exhaustive search infeasible. Consequently, we estimate both the reference optimum and the feasible exploration rate of the search space by randomly sampling 10^{10} points within the domain (the optimum is known analytically only in the unconstrained continuous relaxation).

We then evaluate d-MALIBOO across a broad range of real-world scenarios, spanning private edge systems to public cloud environments. Specifically, we optimize four applications that differ in the complexity of the optimization domain (e.g., dimension, number of possible configurations, and distribution of feasible solutions within the discrete space) and in the nature of the black-box constraints. The differences in problem complexity require different numbers of initial points n_0 and algorithm iterations N for each scenario. These real-world applications are used for both an ablation study of the reparameterization techniques of d-MALIBOO and for comparison with the SOTA approaches.

For all applications, we use constraint values established in previous research [18].

1) *Recipe Transcriber*: A video-processing AI application that identifies recipe ingredients through audio transcription and object detection in video frames. The application comprises five compute-intensive components deployed on AWS EC2 VMs through the OSCAR framework [40]. The optimization vector $\mathbf{x}$ includes the number of cores dedicated to each component, from one up to the maximum that avoids resource saturation. Each execution of the application processes a batch of 100 input videos, each 10 seconds long. The goal of the optimization problem is to identify the configuration that minimizes the *cost* of execution on the cloud while keeping the *execution time* below a maximum threshold T_{max} . We repeat the optimization for $T_{max} \in \{150, 300, 450, 600\}$ seconds.

2) *Query52*: A Spark-based application that executes an interactive query from the TPC-DS industry benchmark¹, representing SQL-like tasks. The application is deployed on Microsoft Azure using the HDInsight service. We use input datasets of varying sizes, and the application can use up to 26 VMs of five possible types. The input vector $\mathbf{x}$ specifies the VM type and number, as well as the memory allocated to each VM. The goal of the optimization is also in this case to minimize the execution *cost* while keeping the maximum *execution time* below T_{max} . Here, we use $T_{max} \in \{265, 335, 405, 645, 2250\}$ seconds.

3) *Stereomatch*: An edge computing image-processing system [41] that computes the disparity between pairs of stereo images captured from the same scene by two cameras, en-

¹<https://www.tpc.org/tpcds/>

abling object depth estimation. It employs adaptive-shape local support windows for each pixel based on color similarity. The input configuration $\mathbf{x}$ includes four parameters: the *number of parallel cores* used, the *color similarity confidence*, the *granularity of disparity hypotheses* to test, and the *length of the support window arm*. We use an input dataset of 40 pairs of images. As in the earlier cases, the objective is to minimize the execution *cost* while meeting an *execution time* constraint, specifically $T_{\max} \in \{8, 10, 12, 17, 40\}$ seconds. The number of possible configurations for this application is significantly larger than the previous cases.

4) *LiGen*: A molecular docking application integrated into the EXSCALATE drug discovery platform [42]. It simulates various ligand-pocket interactions to identify promising docking poses of a ligand within a binding pocket. The quality of the docking solution is assessed by calculating the average Root Mean Square Distance (RMSD) across 100 ligand-protein pairs with known optimal crystal positions. In this optimization scenario, each input configuration $\mathbf{x}$ is defined by eight discrete parameters: six influence the accuracy of the docking algorithm, while the remaining two affect its performance. The OF to minimize is $f(\mathbf{x}) = R^3(\mathbf{x})T(\mathbf{x})$, where $R(\mathbf{x})$ is the RMSD and $T(\mathbf{x})$ is the execution time, subject to the quality constraint $R(\mathbf{x}) \leq R_{\max}$. We consider $R_{\max} \in \{2.0, 2.1, 2.2, 2.45, 2.75\}$ as RMSD thresholds.

Table II summarizes the optimization scenarios and the relevant related quantities.

TABLE II: Real-world optimization scenarios. T denotes the OF evaluation time, R the RMSD.

Application	d	D	$f(\cdot)$	$g(\cdot)$	n_0	N
<i>Recipe Tran.</i>	5	276	cost	$T \leq T_{\max}$	3	10
<i>Query52</i>	2	696	cost	$T \leq T_{\max}$	3	30
<i>Stereomatch</i>	4	18527	cost	$T \leq T_{\max}$	3	60
<i>LiGen</i>	8	491520	$R^3(\mathbf{x})T(\mathbf{x})$	$R \leq R_{\max}$	11	60

B. Methods under comparison

This section describes the experimental setup for our d-MALIBOO algorithm and the SOTA approaches.

The reference configuration for d-MALIBOO is as follows. The GP surrogate model is initialized with a constant mean function. Our ML model of choice for both the constraint and OF prediction models is Ridge Regression, but we also test Random Forest, XGBoost, and Neural Networks. In particular, our reference configuration uses the *indicator* approach. We pre-process training data for the ML models by adding all second-degree polynomial combinations of the original input features. This feature expansion and hyperparameter tuning of the models are performed automatically via aMLLibrary [43]. Preliminary analysis indicates that the test-set Mean Absolute Percentage Error remains consistently below 10% across all tested applications and models. For the ε -greedy approach, we test several values of the ε parameter in the ablation study. For all other experiments, we set $\varepsilon = 0.1$. For the reparameterization method, we choose the RBF kernel with length scale bounds $l_0 = 1.0$ and $l_h = 10.0$, and the Matérn kernel with smoothness parameter bounds $\nu_0 = 2.5$ and $\nu_h = 10.0$. Additionally, we use the UCB as AF with $\lambda_0 = 1.0$ and $\lambda_h = 10.0$, as in [11].

d-MALIBOO runs on a Linux server equipped with a 40-core Intel(R) Xeon(R) CPU at 2.20 GHz and 32 GB of RAM. While the Schwefel test function is evaluated on the same machine, the real-world applications are executed on dedicated machines, as detailed in Section VI-A. Moreover, all competitor approaches run on the same machine as d-MALIBOO, except for BO-PR, as explained below.

We now briefly discuss the settings we use for the alternative literature approaches introduced in Section II-C:

- *MALIBOO*: we use the Matérn kernel (with $\nu = \nu_0$) and the EI AF, like in the original work [18].
- *Discrete-BO*: we use the RBF kernel and the UCB AF, following the original work [11], and adopt the same hyperparameters as d-MALIBOO.
- *SVM-CBO*: we split the total iteration budget between the two phases using the same proportion as [12].
- *BO-PR*: since the algorithm requires a GPU, we run the analyses on a different Linux server equipped with a 20-core Intel(R) Xeon(R) Silver 4114 CPU at 2.20 GHz, 45 GB of memory, and two NVIDIA Quadro P600 GPUs. Without GPUs, the computational time would be prohibitive for the most complex applications.
- *OpenTuner*: we use the default ensemble of algorithms [19].

We recall that d-MALIBOO employs LHS during initialization, whereas MALIBOO, Discrete-BO, SVM-CBO, and OpenTuner rely on random sampling. In contrast, BO-PR uses the Sobol sampling technique [44], which generates samples that are as uniformly distributed as possible over a multi-dimensional parameter space, similar to LHS. The number of initial points n_0 is the same for all algorithms (see Table II).

Finally, to ensure a robust experimental analysis, each configuration is tested $K = 30$ times using different random seeds, resulting in distinct sets of initial points. We release our code on GitHub² and our experimental data on Zenodo³.

C. Evaluation metrics

We illustrate the metrics to evaluate the performance of the optimization algorithms:

- *Mean Absolute Percentage Regret (MAPR)*: the relative deviation between the best solution found and the optimum:

$$MAPR(\hat{\mathbf{f}}, f^*) = \frac{1}{K} \sum_{k=1}^K \left| \frac{\hat{f}_k - f^*}{f^*} \right| \times 100\%$$

where $\hat{\mathbf{f}}$ denotes the vector of the best OF values across repetitions and f^* is the minimum ground-truth of the OF. This metric is a normalized version of the regret, expressed as a percentage relative to the true optimum, and measures the efficiency of an algorithm in finding high-quality configurations. Unlike absolute regret, MAPR facilitates direct comparison across applications.

- *Feasible exploration rate*: the proportion of feasible configurations explored relative to the total number of evaluated

²<https://github.com/RobertoSala98/d-MALIBOO.git>

³<https://doi.org/10.5281/zenodo.14831044>

configurations, averaged over different repetitions. It measures the ability of the algorithm to identify, explore, and effectively exploit the feasible region across iterations.

- *Total wall-clock optimization time*: the average running time of the algorithm over different repetitions. This measure includes both the *OF evaluation time* (i.e., the execution time of the application under a given configuration) and the *algorithm processing time* (i.e., the time required to identify the next candidate configuration). Although the processing time is generally expected to represent a negligible portion of the total time, this is not always the case, as it will be discussed later. Indeed, time efficiency is also an important factor when evaluating the performance of an algorithm.

To assess performance differences between algorithms, we employ the Mann-Whitney test [45], a nonparametric statistical test used to determine whether two independent samples come from the same distribution, i.e., if they are homogeneous.

VII. EXPERIMENTAL RESULTS

This section presents the results of our experimental campaign. Section VII-A reports the results of the ablation study and the scalability analysis on the Schwefel function, while Section VII-B presents the ablation study of the reparameterization technique and the comparison with SOTA approaches on the real-world applications. Section VII-C examines the proposed approach in light of the experimental findings.

A. Evaluation on Schwefel function

We now present an ablation study of d-MALIBOO using the Schwefel function as the OF to optimize, varying one parameter at a time from the reference configuration introduced in Section VI-B to assess the robustness of the algorithm under different settings. Next, we evaluate the scalability of our approach by varying the dimension d of the discrete domain.

1) *Ablation study*: We run the d-MALIBOO algorithm by considering as reference configuration the one described in Section VI-B and reported in the first line of Table III, and by varying the following parameters:

- *indicator* vs *probability* approach (for both the objective and constraint functions);
- ε value for the ε -greedy approach;
- ML model to estimate the objective and constraint functions, choosing between Random Forest (RF), Neural Network (NN), and XGBoost.

Table III collects the results of the ablation study in terms of MAPR, feasible exploration rate, and total optimization time, averaged over the $K = 30$ repetitions.

We observe that the variation in MAPR is not statistically significant for any configuration. This is confirmed by the Mann-Whitney test at the 5% significance level, performed for all pairs of possible settings (see Table XIV in Appendix C). In rare cases (with RF as ML model), the p-value reaches 7%, which we do not consider enough to reject the null hypothesis of equal MAPR distributions. The largest differences in performance are observed when varying the ML models. In particular, the RF model achieves the lowest MAPR, but

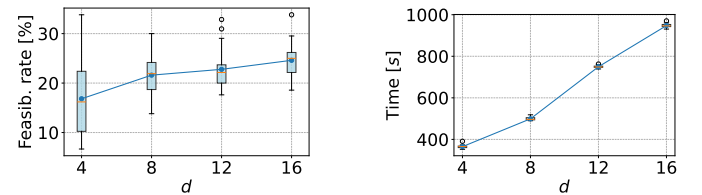
TABLE III: Results of the ablation study. The first row represents the reference d-MALIBOO configuration.

ML appr.	ε	ML model	MAPR [%]	Feas. [%]	Time [s]
indicator	0.1	Ridge	187.47	21.60	499.01
probability	0.1	Ridge	196.17	24.08	506.39
indicator	0.01	Ridge	196.32	20.67	506.10
indicator	0.025	Ridge	184.89	21.76	501.07
indicator	0.05	Ridge	188.35	20.54	499.93
indicator	0.2	Ridge	195.09	23.79	486.40
indicator	0.1	RF	178.65	17.71	685.26
indicator	0.1	NN	185.52	37.51	1124.86
indicator	0.1	XGBoost	191.33	21.08	85909.46

its feasible exploration rate falls below the random chance baseline of 20%. In contrast, the NN achieves the highest feasibility, though it requires more than twice the execution time compared to Ridge Regression. XGBoost is also significantly slower. Henceforth, we adopt Ridge Regression as the ML model in our experiments, as it provides a favorable trade-off across all metrics. For the sake of completeness, we also optimize the Schwefel function with the alternative SOTA approaches of Section VI-B. The results are reported in Table XIII in Appendix B and show that the MAPR achieved (in the range 179.84%–232.56%) is in line with d-MALIBOO.

Note that reparameterization never triggers in our experiments with the test function due to the large number of points in the domain. This allows us to study the effect of reparameterization independently of the other features of the algorithm (Section VII-B).

2) *Scalability analysis*: As previously discussed, we now vary the dimension of the discrete domain, $d = 4, 8, 12, 16$, while maintaining a constant 20% feasibility rate of the search space. Figure 2 shows the feasible exploration rate and total optimization time of d-MALIBOO for each value of d . In these experiments, we use the d-MALIBOO reference configuration presented in Section VI-B.



(a) Feasible exploration rate.

(b) Total optimization time.

Fig. 2: Scalability analysis of d-MALIBOO.

The feasible exploration rate (Figure 2a) exhibits a slight increasing trend across dimensions, although the differences are not statistically significant. The total optimization time (Figure 2b) increases with d , primarily due to the longer training times of the ML models. Nevertheless, the overall optimization time is largely dominated by updates of the surrogate model, whose complexity depends mainly on the number of observed points n . These results indicate that d-MALIBOO is scalable with respect to the domain dimension d , as in real-world scenarios, the wall-clock time would be dominated by the evaluation of the OF.

B. Evaluation on real-world applications

This section presents the numerical results of our d-MALIBOO algorithm on the real-world scenarios. Specifically,

we first provide an ablation study of the reparameterization techniques, followed by a comparison with the SOTA methods.

1) *Ablation study of reparameterization strategies:* We compare the d-MALIBOO reparameterization strategies presented in Section V across the four real-world optimization scenarios. The ML model approaches we adopt are the best-performing ones identified in our previous work [28], namely the *indicator* for the constraints and the *probability* for the OF.

Figure 3 displays the regrets obtained by d-MALIBOO with the four reparameterization strategies. Each subfigure corresponds to a specific optimization scenario and reports a set of boxplots for each upper-bound value of the constraint function. Tables IV, V, VI, and VII summarize the average MAPR and feasibility exploration rate across the K repetitions. For each threshold, the configuration with the lowest regret is highlighted in bold, and the second-best one in italics.

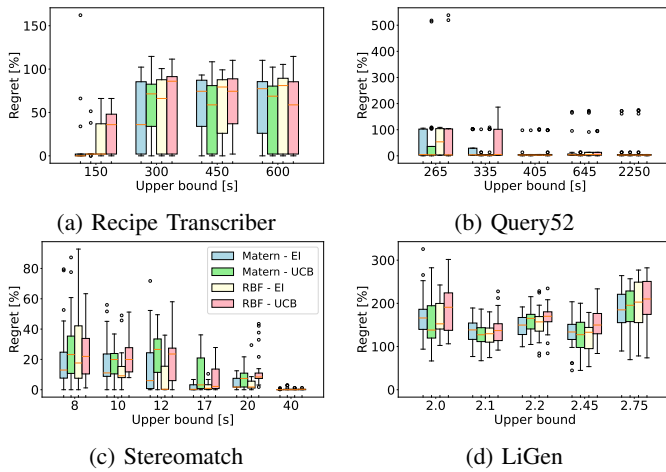


Fig. 3: Box plots of the regrets achieved by d-MALIBOO variants for each optimized application.

The results show that no configuration consistently outperforms the others across all applications or thresholds. Specifically, for the *Recipe Transcriber* application, configurations using the *Matérn* kernel appear to be more effective than those using *RBF* (see Figure 3a and Table IV). In *Query52*, the configuration combining the *Matérn* kernel with the *UCB* AF performs best, except under the strictest constraint. However, this setup consistently achieves the lowest percentage of feasible configurations explored (see Figure 3b and Table V). For *Stereomatch*, the two configurations using the *EI* AF yield the best results in terms of both regret and the percentage of feasible configurations explored (see Figure 3c and Table VI). Moreover, for almost all threshold values except the strictest one, these two configurations show lower variability than those using the *UCB* AF. Finally, in *LiGen*, all configurations demonstrate comparable average performance. However, the configuration using the *RBF* kernel with the *EI* AF proves to be the most effective in terms of MAPR across all thresholds (see Figure 3d and Table VII).

The Mann-Whitney test confirms the homogeneity of performance across the reparameterization strategies, with only a few exceptions (see Tables XV, XVI, XVII, and XVIII in Appendix C). For instance, in the *LiGen* scenario, the con-

figuration using the *RBF* GP kernel with the *UCB* AF shows significantly worse performance than the others. Furthermore, while the combination of *Matérn* kernel and *UCB* AF achieves better results for applications with smaller domains, the configuration with *RBF* kernel and *EI* AF proves more effective for the more challenging optimization problems. For this reason, in the following analysis, we use the latter configuration to compare d-MALIBOO with the SOTA approaches.

2) *Comparison with the SOTA:* We now compare the results of d-MALIBOO with the SOTA approaches described in Section II-C. Specifically, we analyze the wall-clock time performance of the algorithms, followed by a comparison of MAPR and feasible exploration rates.

a) *Time efficiency:* Table VIII reports the average OF evaluation time (i.e., the time to execute the application under study and target of the optimization process), the processing time per iteration T_{iter} , and the total optimization time T_{tot} (i.e., the time to perform the N iterations) for all scenarios.

We observe that all algorithms, except BO-PR, exhibit similar T_{iter} , which is significantly negligible compared to the average OF evaluation time. Conversely, SVM-CBO is extremely slow in terms of total optimization time, while OpenTuner proves to be the most efficient on *Stereomatch*. d-MALIBOO, MALIBOO, and Discrete-BO show comparable T_{tot} , with MALIBOO being faster on *LiGen*. Furthermore, BO-PR is inefficient across all applications in terms of T_{tot} . The performance gap with d-MALIBOO is more pronounced in the most complex applications, namely *Stereomatch* ($4.62\times$ slower) and *LiGen* ($3.23\times$ slower). Notably, in these two cases, T_{iter} exceeds the average OF evaluation time, reaching up to five times higher in *Stereomatch*.

The impact of the algorithmic overhead is further highlighted in Figure 4, which shows the MAPR trends of the different algorithms throughout the optimization of *LiGen*. Note that the plot exhibits occasional MAPR increases because it represents the average over experimental repetitions, and the first feasible point in each run may be identified at different times. We observe that the wall-clock total execution time

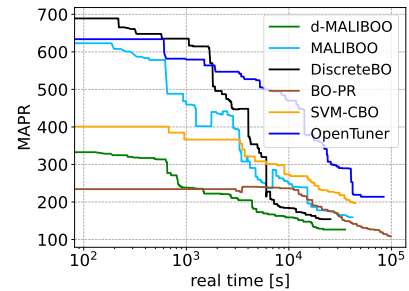


Fig. 4: MAPR trend of all the algorithms on *LiGen*, with constraint $R(x) \leq 2.1$.

of BO-PR is approximately 3-4 times longer than that of d-MALIBOO for achieving a similar MAPR. These times make BO-PR impractical for real-world applications, as additional costs arise from idle machines during the optimization process.

For additional insights, we provide a more detailed discussion in Appendix A. We compare the performance of the considered optimization algorithms in terms of real-time efficiency

TABLE IV: Performance of d-MALIBOO variants for *Recipe Transcriber*.

Kernel	AF	$T(x) \leq 150s$		$T(x) \leq 300s$		$T(x) \leq 450s$		$T(x) \leq 600s$	
		MAPR[%]	feas.[%]	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]
Matérn	EI	29.59	12.09	41.69	78.57	59.36	95.88	58.63	100.00
Matérn	UCB	13.92	5.22	57.20	71.15	49.79	94.78	48.9	100.00
RBF	EI	25.75	12.64	52.89	76.10	60.66	95.88	58.21	100.00
RBF	UCB	36.75	6.04	61.93	71.15	62.48	94.51	49.1	100.00

TABLE V: Performance of d-MALIBOO variants for *Query52*.

Kernel	AF	$T(x) \leq 265s$		$T(x) \leq 335s$		$T(x) \leq 405s$		$T(x) \leq 645s$		$T(x) \leq 2250s$	
		MAPR[%]	feas.[%]	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]
Matérn	EI	32.29	30.09	28.07	45.67	7.09	65.91	19.70	85.82	21.64	100.00
Matérn	UCB	58.70	25.87	6.77	41.99	6.54	61.36	3.91	80.41	3.84	100.00
RBF	EI	53.99	29.55	21.04	46.10	13.76	63.64	25.35	87.01	27.66	100.00
RBF	UCB	65.76	22.94	35.60	43.29	17.07	64.83	14.78	82.79	3.39	100.00

TABLE VI: Performance of d-MALIBOO variants for *Stereomatch*.

Kernel	AF	$T(x) \leq 8s$		$T(x) \leq 10s$		$T(x) \leq 12s$		$T(x) \leq 17s$		$T(x) \leq 20s$		$T(x) \leq 40s$	
		MAPR[%]	feas.[%]	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]
Matérn	EI	21.11	39.63	17.16	55.73	14.57	71.03	1.64	82.77	3.75	84.13	0.09	82.31
Matérn	UCB	26.52	28.51	17.28	55.61	23.54	67.01	9.33	81.63	7.31	84.18	0.41	84.81
RBF	EI	24.43	36.90	13.24	59.41	8.44	73.19	2.05	82.71	3.92	84.81	0.21	85.43
RBF	UCB	22.30	33.16	21.84	59.07	21.12	67.80	8.57	83.39	12.70	87.81	0.23	86.17

TABLE VII: Performance of d-MALIBOO variants for *LiGen*.

Kernel	AF	$R(x) \leq 2.0$		$R(x) \leq 2.1$		$R(x) \leq 2.2$		$R(x) \leq 2.45$		$R(x) \leq 2.75$	
		MAPR[%]	feas.[%]	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]
Matérn	EI	171.10	16.35	136.10	33.45	146.65	46.98	131.85	84.10	182.64	95.77
Matérn	UCB	157.72	17.35	128.16	35.46	159.42	51.91	128.00	80.23	192.74	94.22
RBF	EI	164.31	16.50	126.62	34.36	153.52	50.45	121.71	82.55	196.43	95.82
RBF	UCB	188.32	17.56	140.66	35.56	166.77	52.57	154.41	81.04	204.41	93.46

TABLE VIII: Average OF evaluation time, processing time per iteration (T_{iter}) and wall-clock total optimization time (T_{tot}) across different applications. All values are in seconds.

avg T_{eval}	Recipe Transcriber 227.14		Query52 595.84		Stereomatch 12.81		LiGen 547.70	
	T_{iter}	T_{tot}	T_{iter}	T_{tot}	T_{iter}	T_{tot}	T_{iter}	T_{tot}
d-MALIBOO	0.10	3160.82	0.54	17568.07	0.26	976.50	3.56	30754.30
MALIBOO	0.08	3184.90	0.09	16072.80	0.41	983.28	7.76	21468.00
Discrete-BO	0.06	3337.01	0.23	17737.64	0.48	1903.71	1.15	23820.81
SVM-CBO	0.53	1.09e5	0.13	5.11e5	0.32	8.63e4	1.27	3.31e6
BO-PR	49.51	3898.92	483.43	24066.61	58.73	4512.00	1368.43	99209.78
OpenTuner	0.08	3595.99	0.72	25310.30	0.04	711.38	1.07	41383.77

highlighting that under a fixed-time budget, d-MALIBOO is both more reliable in finding feasible configurations and achieves a better MAPR on average (Figures 7, 8, 9 and 10). In particular, d-MALIBOO exhibits a faster MAPR decrease than the SOTA approaches. Moreover, its MAPR remains close to those of the best-performing approaches even when it does not achieve the lowest value.

b) Iteration efficiency: We now illustrate the MAPR and feasible exploration rate of the configurations explored by all algorithms across the four applications. Figure 5 presents boxplots of the best regrets obtained for each application. A single orange line indicates that the entire distribution of regrets is concentrated near the median, whereas a blank space indicates that the algorithm did not visit any feasible solution. Tables IX, X, XI, and XII report the measured averages.

In the *Recipe Transcriber* scenario, the relatively small number of algorithm iterations limits the extent of exploration. Results in Figure 5a and Table IX show that MALIBOO achieves the lowest MAPR (15–60%), while d-MALIBOO explores the highest number of feasible configurations. Note that the optimum of these problems always lies in the interior of the feasible domain rather than on its boundary. As a result, the constraint is inactive, allowing the unconstrained

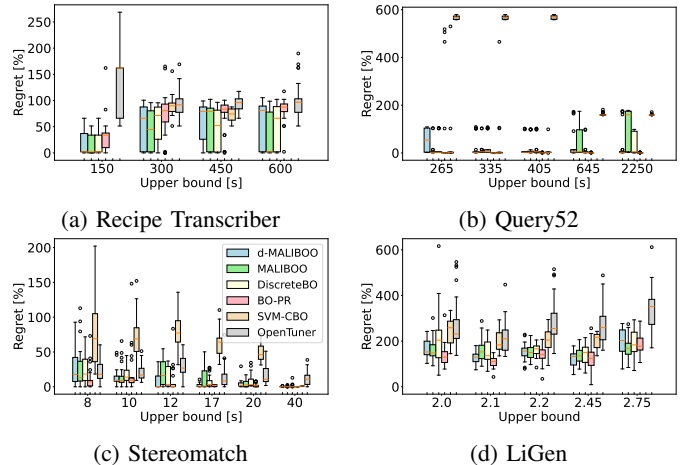


Fig. 5: Box plots of the regrets achieved by d-MALIBOO and SOTA across the reference applications.

technique Discrete-BO to perform comparably to its competitors. d-MALIBOO consistently outperforms BO-PR, achieving a MAPR of 54–82%. OpenTuner achieves 90–140% of MAPR across all threshold values, whereas SVM-CBO exhibits a very low feasible exploration rate, finding feasible configurations in

TABLE IX: Comparison of average metrics in *Recipe Transcriber*.

Algorithm	$T(x) \leq 150s$		$T(x) \leq 300s$		$T(x) \leq 450s$		$T(x) \leq 600s$	
	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]
d-MALIBOO	25.75	12.64	52.89	76.10	60.66	95.88	58.21	100.00
MALIBOO	15.47	9.62	44.41	77.75	59.15	94.23	34.17	100.00
Discrete-BO	26.14	6.32	54.65	70.60	45.39	93.96	54.05	100.00
BO-PR	54.72	10.44	74.08	71.98	73.85	93.68	82.47	100.00
SVM-CBO	33.96	0.27	92.59	51.65	76.22	15.66	inf	0.00
OpenTuner	140.76	5.77	93.16	62.09	93.93	95.60	101.50	100.00

TABLE X: Comparison of average metrics in *Query52*.

Algorithm	$T(x) \leq 265s$		$T(x) \leq 335s$		$T(x) \leq 405s$		$T(x) \leq 645s$		$T(x) \leq 2250s$	
	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]
d-MALIBOO	53.99	29.55	21.04	46.10	13.76	63.64	25.35	87.01	27.66	100.00
MALIBOO	10.82	21.97	12.12	42.21	20.67	63.85	47.42	85.93	109.21	100.00
Discrete-BO	17.65	17.42	23.11	35.82	17.90	53.46	10.34	75.22	30.01	100.00
BO-PR	57.08	81.39	0.00	87.45	0.37	89.50	0.31	96.75	0.44	100.00
SVM-CBO	50.32	48.59	63.28	43.72	18.86	29.33	inf	0.00	inf	0.00
OpenTuner	566.17	14.72	567.50	47.40	568.21	48.59	161.66	59.31	159.71	100.00

TABLE XI: Comparison of average metrics in *Stereomatch*.

Algorithm	$T(x) \leq 8s$		$T(x) \leq 10s$		$T(x) \leq 12s$		$T(x) \leq 17s$		$T(x) \leq 20s$		$T(x) \leq 40s$	
	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]
d-MALIBOO	24.43	36.90	13.24	59.41	8.44	73.19	2.05	82.71	3.92	84.81	0.21	85.43
MALIBOO	26.14	43.71	14.33	64.85	18.90	73.24	10.76	83.62	4.51	86.56	0.20	82.99
Discrete-BO	23.71	14.23	17.21	29.08	14.67	39.34	7.55	56.41	7.95	61.28	0.80	73.47
BO-PR	10.57	34.98	18.13	61.68	7.14	74.55	1.76	86.05	1.83	89.23	0.00	90.42
SVM-CBO	92.40	12.93	75.49	35.77	87.30	43.99	73.69	26.30	68.93	26.47	1.16	6.07
OpenTuner	22.11	58.79	21.02	77.15	27.48	81.41	14.32	88.78	17.86	87.53	11.74	95.18

TABLE XII: Comparison of average metrics in *LiGen*.

Algorithm	$R(x) \leq 2.0$		$R(x) \leq 2.1$		$R(x) \leq 2.2$		$R(x) \leq 2.45$		$R(x) \leq 2.75$	
	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]	MAPR[%]	feas.[%]
d-MALIBOO	164.31	16.50	126.62	34.36	153.52	50.45	121.71	82.55	196.43	95.82
MALIBOO	162.61	13.58	159.39	24.80	151.73	40.90	137.02	76.61	169.38	95.93
Discrete-BO	236.84	8.65	154.19	21.13	161.35	32.24	140.48	73.19	189.77	94.62
BO-PR	137.36	15.59	108.35	33.00	139.35	56.54	125.18	91.15	190.12	99.45
SVM-CBO	256.31	16.65	197.20	39.69	210.34	52.97	206.56	24.14	inf	0.00
OpenTuner	275.94	31.99	213.90	49.90	283.59	64.44	269.02	83.55	343.77	97.94

only 3% of cases under the strictest constraint and 0% under the largest. This behavior occurs because, when only a few initial points are evaluated, they may all be feasible or all unfeasible, which limits the ability of the SVM classifier to estimate the boundary of the feasible region accurately.

In the *Query52* scenario, the primary challenge for optimization is the irregularity of the OF, whose values span multiple orders of magnitude. The results (see Figure 5b and Table X) show that BO-PR is the best-performing algorithm across almost all thresholds, except the strictest one. MALIBOO achieves low regret for the strictest constraints (10-12%), but this increases with the threshold value, reaching over 100%. In contrast, d-MALIBOO demonstrates more stable performance, with a peak MAPR of 54% for the strictest constraint and a higher feasible exploration rate compared to MALIBOO. Discrete-BO performs similarly to d-MALIBOO because the optimum also lies in the interior of the domain in these cases. SVM-CBO struggles with the feasible region estimation, and OpenTuner is unable to find a near-optimal solution.

Stereomatch is one of our most challenging case studies due to the large number of possible configurations. The results (see Figure 5c and Table XI) show that BO-PR achieves the best performance in terms of regret, with an average MAPR of 11% for $T(x) \leq 8s$ and less than 2% for the highest threshold values. d-MALIBOO achieves results similar to BO-PR but in significantly less time. Our proposed algorithm outperforms both Discrete-BO and MALIBOO, although MALIBOO

exhibits a slightly higher percentage of feasible configurations explored under the strictest constraints. Additionally, both MALIBOO and Discrete-BO show greater variability in MAPR. Unlike in previous scenarios, OpenTuner performs only slightly worse than d-MALIBOO, particularly for larger bound values. In contrast, SVM-CBO performs significantly worse than all other approaches, achieving high MAPR and low feasibility across almost all bound values.

LiGen is the most complex scenario in this experimental campaign due to the even larger size of its domain. Results in Figure 5d and Table XII show that OpenTuner and SVM-CBO are the worst-performing algorithms in terms of MAPR, both achieving 200% or more across all threshold values. Discrete-BO also exhibits a high MAPR for the strictest constraint and a lower feasible exploration rate compared to d-MALIBOO across all thresholds. Furthermore, d-MALIBOO outperforms MALIBOO on average by 5% in MAPR and 10% in feasible exploration rate. Figure 4 shows that d-MALIBOO achieves a similar performance to MALIBOO in fewer iterations, allowing it to further improve the proposed configurations. BO-PR achieves a lower MAPR and a feasible exploration rate comparable to d-MALIBOO, but requires more than three times the total computational time (see Table VIII).

Finally, Tables XIX, XX, XXI, and XXII (in Appendix C) report the results of the Mann-Whitney test comparing d-MALIBOO with the SOTA approaches. The results show that MALIBOO is comparable to d-MALIBOO in terms of

MAPR for certain bounds across the applications, particularly on *Recipe Transcriber*, while Discrete-BO exhibits performance homogeneous to that of our framework mainly in this scenario. The difference between d-MALIBOO and BO-PR is significant in the simpler scenarios, but becomes less pronounced in the more complex ones. Conversely, SVM-CBO and OpenTuner almost always exhibit MAPR distributions that differ significantly from that of d-MALIBOO.

C. Discussion

In the experimental campaign, we first show that d-MALIBOO is robust with respect to its parameter settings. In particular, variations in the choice of GP kernel, acquisition function, ML models, and the ε hyperparameter do not cause significant differences in MAPR.

In the comparison with alternative SOTA approaches, the four real-world optimization scenarios we present pose different challenges, with some algorithms performing well in certain settings but poorly in others. In contrast, d-MALIBOO demonstrates consistent and reliable performance across all measured metrics: regret, feasible exploration rate, and total wall-clock optimization time. MALIBOO shows more inconsistent performance, with a higher MAPR and a lower feasible exploration rate in the most complex scenarios. Discrete-BO exhibits a much lower feasible exploration rate than d-MALIBOO in scenarios where the optimum lies on the boundary of the feasible region. SVM-CBO is consistently outperformed by d-MALIBOO across all metrics due to its inefficient management of observed out-of-bounds configurations. BO-PR achieves the best MAPR, but its per-iteration processing time exceeds the average OF evaluation time, which makes it less practical for real-world applications. OpenTuner consistently finds feasible configurations, but d-MALIBOO always outperforms it in terms of MAPR.

These results suggest that the components of d-MALIBOO are well-suited to tackle the problems under study. The BO approach enables the algorithm to achieve a low MAPR within few iterations, while the ML models assist in estimating the feasible region, which improves the feasible exploration rate, and in guiding the search for the optimum, which enhances the exploitation capability of the algorithm. Furthermore, the ε -greedy strategy promotes exploration, while the reparameterization module dynamically adjusts the surrogate model to avoid repeating already observed points. Overall, the experimental results show that d-MALIBOO outperforms the state of the art in the tested scenarios in terms of the quality of optimal configurations detected (MAPR), extent of domain exploration (feasible exploration rate), and time efficiency (wall-clock optimization time). Overall, compared to the five SOTA approaches, d-MALIBOO achieves up to 142% lower regret, explores up to 39% more feasible configurations, and reduces wall-clock optimization time by a factor of 3–5.

Finally, we outline some limitations of our methodology. First, d-MALIBOO is inherently sequential and is not suitable for scenarios where the OF can be evaluated simultaneously on multiple servers. Integrations with methods that allow parallel evaluations, both with [46] and without BO [39], may

be considered. Second, inaccurate ML models could lead to imprecise estimates of the feasible region and, therefore, low feasible exploration rates. Nevertheless, d-MALIBOO identified at least one feasible configuration in all tested scenarios.

VIII. CONCLUSION

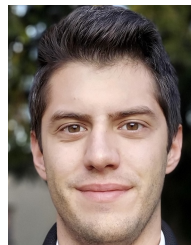
This work presents d-MALIBOO, a BO algorithm for constrained optimization in multidimensional discrete domains. Our framework is tailored for scenarios involving black-box OFs and constraints and leverages ML techniques to estimate the feasible region and guide the AF toward the optimal solution. In addition, reparameterization techniques dynamically adjust the surrogate model to prevent evaluating configurations that have already been explored. Our experimental results demonstrate that, compared to SOTA approaches, d-MALIBOO: 1) is robust to parameter settings; 2) finds higher-quality configurations in the optimization process; 3) explores the feasible portion of the optimization domain more effectively; and 4) reduces wall-clock optimization time.

Future work will focus on extending our technique to mixed-variable scenarios and on developing parallel optimization strategies to further enhance the efficiency of discrete BO.

REFERENCES

- [1] M. Pournader *et al.*, “Artificial intelligence applications in supply chain management,” *International Journal of Production Economics*, vol. 241, p. 108250, 2021.
- [2] Z. Zhang *et al.*, “Explainable artificial intelligence applications in cyber security: State-of-the-art in research,” *IEEE Access*, vol. 10, pp. 93104–93139, 2022.
- [3] J. Yin *et al.*, “Role of artificial intelligence applications in real-life clinical practice: Systematic review,” *Journal of medical Internet research*, vol. 23, no. 4, e25759, 2021.
- [4] O. Alipourfard *et al.*, “Cherrypick: Adaptively unearthing the best cloud configurations for big data analytics,” in *14th USENIX Symposium on Networked Systems Design and Implementation*, 2017, pp. 469–482.
- [5] R.-R. Griffiths and J. M. Hernández-Lobato, “Constrained Bayesian optimization for automatic chemical design using variational autoencoders,” *Chemical science*, vol. 11, no. 2, pp. 577–586, 2020.
- [6] M. Injadat *et al.*, “Bayesian optimization with machine learning algorithms towards anomaly detection,” in *IEEE Global Communications Conference*, 2018, pp. 1–6.
- [7] K. Kandasamy *et al.*, “Neural architecture search with Bayesian optimisation and optimal transport,” in *Advances in Neural Information Processing Systems 31*, 2018, pp. 2020–2029.
- [8] Z. Zhao *et al.*, “A multimodel edge computing offloading framework for deep-learning application based on Bayesian optimization,” *IEEE Internet Things J.*, vol. 10, no. 20, pp. 18387–18399, 2023.
- [9] J. Mockus, “On Bayesian methods for seeking the extremum,” in *Optimization Techniques*, vol. 27, 1974, pp. 400–404.
- [10] P. I. Frazier, “A tutorial on Bayesian optimization,” *CoRR*, vol. abs/1807.02811, 2018.
- [11] P. Luong *et al.*, “Bayesian optimization with discrete variables,” in *Advances in Artificial Intelligence*, J. Liu and J. Bailey, Eds., vol. 11919, 2019, pp. 473–484.
- [12] A. Candelieri, “Sequential model based optimization of partially defined functions under unknown constraints,” *J. Glob. Optim.*, vol. 79, no. 2, pp. 281–303, 2021.
- [13] S. Daulton *et al.*, “Bayesian optimization over discrete and mixed spaces via probabilistic reparameterization,” in *Advances in Neural Information Processing Systems 35*, 2022.
- [14] E. C. Garrido-Merchán and D. Hernández-Lobato, “Dealing with categorical and integer-valued variables in Bayesian optimization with Gaussian processes,” *Neurocomputing*, vol. 380, pp. 20–35, 2020.

- [15] J. Yan *et al.*, “Bayesian optimization for online management in dynamic mobile edge computing,” *IEEE Trans. Wirel. Commun.*, vol. 23, no. 4, pp. 3425–3436, 2024.
- [16] M. A. Gelbart *et al.*, “Bayesian optimization with unknown constraints,” in *Proceedings of the Thirtieth Conference on Uncertainty in Artificial Intelligence*, 2014, pp. 250–259.
- [17] J. M. Hernández-Lobato *et al.*, “A general framework for constrained Bayesian optimization using information-based search,” *J. Mach. Learn. Res.*, vol. 17, 160:1–160:53, 2016.
- [18] B. Guindani *et al.*, “Integrating Bayesian optimization and machine learning for the optimal configuration of cloud systems,” *IEEE Trans. Cloud Comput.*, vol. 12, no. 1, pp. 277–294, 2024.
- [19] J. Ansel *et al.*, “Opentuner: An extensible framework for program autotuning,” in *International Conference on Parallel Architectures and Compilation*, 2014, pp. 303–316.
- [20] J. Bergstra *et al.*, “Hyperopt: A python library for optimizing the hyperparameters of machine learning algorithms,” in *Proceedings of the 12th Python in Science Conference*, 2013, pp. 13–19.
- [21] D. Golovin *et al.*, “Google vizier: A service for black-box optimization,” in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2017, pp. 1487–1495.
- [22] T. Miyazaki *et al.*, “Bayesian optimization of HPC systems for energy efficiency,” in *33rd International Conference on High Performance Computing*, vol. 10876, 2018, pp. 44–62.
- [23] H. Menon *et al.*, “Auto-tuning parameter choices in HPC applications using Bayesian optimization,” in *IEEE International Parallel and Distributed Processing Symposium*, 2020, pp. 831–840.
- [24] M. Diessner *et al.*, “Investigating Bayesian optimization for expensive-to-evaluate black box functions: Application in fluid dynamics,” *Frontiers Appl. Math. Stat.*, vol. 8, 2022.
- [25] S. Krishnamoorthy *et al.*, “Diffusion models for black-box optimization,” in *International Conference on Machine Learning*, vol. 202, 2023, pp. 17 842–17 857.
- [26] X. Song *et al.*, “Position: Leverage foundational models for black-box optimization,” in *Forty-first International Conference on Machine Learning*, 2024.
- [27] Z. Li *et al.*, “Diffusion model for data-driven black-box optimization,” *CoRR*, 2024.
- [28] R. Sala *et al.*, “D-maliboo: A Bayesian optimization framework for dealing with discrete variables,” in *32nd International Conference on Modeling, Analysis and Simulation of Computer and Telecommunication Systems*, 2024, pp. 1–8.
- [29] R. Baptista and M. Poloczek, “Bayesian optimization of combinatorial structures,” in *Proceedings of the 35th International Conference on Machine Learning*, vol. 80, 2018, pp. 471–480.
- [30] B. X. Ru *et al.*, “Bayesian optimisation over multiple continuous and categorical inputs,” in *Proceedings of the 37th International Conference on Machine Learning*, vol. 119, 2020, pp. 8276–8285.
- [31] D. Eriksson and M. Poloczek, “Scalable constrained Bayesian optimization,” in *The 24th International Conference on Artificial Intelligence and Statistics*, vol. 130, 2021, pp. 730–738.
- [32] R. Michael *et al.*, “A continuous relaxation for discrete Bayesian optimization,” *CoRR*, vol. abs/2404.17452, 2024.
- [33] X. Sun *et al.*, “On efficient variants of segment anything model: A survey,” *Int. J. Comput. Vis.*, vol. 133, no. 10, pp. 7406–7436, 2025.
- [34] C. E. Rasmussen and C. K. I. Williams, *Gaussian processes for machine learning*. 2006.
- [35] V. Mnih *et al.*, “Human-level control through deep reinforcement learning,” *Nat.*, vol. 518, no. 7540, pp. 529–533, 2015.
- [36] R. Martinez-Cantin, “Bayesian optimization with adaptive kernels for robot control,” in *IEEE International Conference on Robotics and Automation*, 2017, pp. 3350–3356.
- [37] W.-L. Loh, “On latin hypercube sampling,” *The Annals of Statistics*, vol. 24, no. 5, pp. 2058–2080, 1996.
- [38] M. Laguna and R. Marti, “Experimental testing of advanced scatter search designs for global optimization of multimodal functions,” *Journal of Global Optimization*, vol. 33, no. 2, pp. 235–255, 2005.
- [39] R. Sala *et al.*, “An artificial bee colony algorithm with machine learning for constrained optimization in HPC,” in *33rd International Conference on Modeling, Analysis and Simulation of Computer and Telecommunication Systems*, 2025, pp. 1–8.
- [40] S. Risco *et al.*, “Serverless workflows for containerised applications in the cloud continuum,” *J. Grid Comput.*, vol. 19, no. 3, p. 30, 2021.
- [41] E. Paone *et al.*, “An exploration methodology for a customizable opencv stereo-matching application targeted to an industrial multi-cluster architecture,” in *Proceedings of the 10th International Conference on Hardware/Software Codesign and System Synthesis*, 2012, pp. 503–512.
- [42] D. Gadioli *et al.*, “EXSCALATE: an extreme-scale virtual screening platform for drug discovery targeting polypharmacology to fight sars-cov-2,” *IEEE Trans. Emerg. Top. Comput.*, vol. 11, no. 1, pp. 170–181, 2023.
- [43] B. Guindani *et al.*, “Amllibrary: An automl approach for performance prediction,” in *Proceedings of the 37th ECMS International Conference on Modelling and Simulation*, 2023, pp. 215–221.
- [44] A. Saltelli *et al.*, “Variance based sensitivity analysis of model output. design and estimator for the total sensitivity index,” *Comput. Phys. Commun.*, vol. 181, no. 2, pp. 259–270, 2010.
- [45] H. B. Mann and D. R. Whitney, “On a test of whether one of two random variables is stochastically larger than the other,” *The annals of mathematical statistics*, pp. 50–60, 1947.
- [46] B. Guindani *et al.*, “Efficient parameter tuning for a structure-based virtual screening HPC application,” *J. Parallel Distributed Comput.*, vol. 202, p. 105 087, 2025.



Roberto Sala received the M.Sc. degree in Mathematical Engineering from Politecnico di Milano, Italy, in 2023. He is currently a Ph.D. student in Computer Science and Engineering at the same institution. His research interests focus on modeling and performance management of HPC-cloud-edge systems, Bayesian optimization methods for auto-tuning, and Swarm Intelligence algorithms.



Bruno Guindani is a post-doctoral researcher in Computer Science at the Department of Electronics, Information, and Bioengineering at Politecnico di Milano, Italy. He received his M.Sc. degree in Mathematical Engineering in 2021 and his Ph.D. degree in Computer Science and Engineering in 2024 from the same University. His research interests include Bayesian models and Machine Learning techniques for the optimization of cloud and HPC systems.



Alessandra Guglielmi received her Ph.D. in Mathematics from the University of Milano, Italy, in 1997. She is Full Professor in Statistics at Politecnico di Milano. Her recent research interests are Bayesian mixture models for density estimation and clustering and Bayesian Optimization. From the application point of view, her work concerns statistics for medicine, environmental applications, Bayesian Optimization for cloud systems, and big data applications.



Danilo Ardagna is Associate Professor at the Dipartimento di Elettronica Informazione and Bioingegneria at Politecnico di Milano. He received a Ph.D. degree in computer engineering in 2004 from Politecnico di Milano from which he also graduated in December 2000. His work focuses on the design, prototype and evaluation of optimization algorithms for resource management of cloud computing and big data systems.

APPENDIX

For the sake of space, this Appendix provides additional material related to the experimental results presented in Section VII. Specifically, Section A discusses the wall-clock optimization time of d-MALIBOO and the SOTA approaches, expanding on the analysis in Section VII-B. Section B presents the results of the SOTA approaches on the *Schwefel* function. Finally, Section C reports the results of the Mann–Whitney tests conducted throughout the work.

APPENDIX A

WALL-CLOCK OPTIMIZATION TIME ANALYSIS

In this Appendix, we provide further insights into the wall-clock optimization time of our d-MALIBOO algorithm and the SOTA approaches used for comparison. First, Figure 6 illustrates the MAPR trend for a single representative run of all algorithms as a function of wall-clock time. Notably, each algorithm exhibits a monotonically decreasing MAPR once the first feasible configuration is identified. However, since some algorithms do not sample a feasible configuration during the initialization phase, averaging over $K = 30$ runs produces the trend shown in Figure 4.

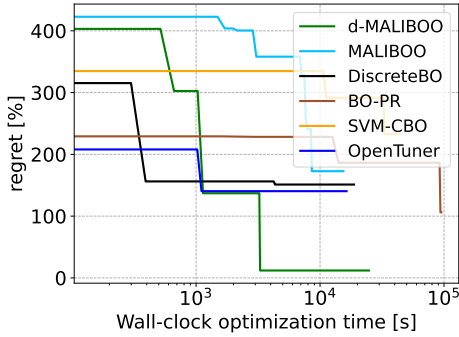


Fig. 6: Comparison of all algorithms for a single experiment on *LiGen* with the constraint $R(\mathbf{x}) \leq 2.45$.

Furthermore, Figures 7, 8, 9, and 10 illustrate the MAPR trends for each real-world application, under the following two assumptions. First, we set the maximum wall-clock optimization time to 15 times the average evaluation time of the OF. This threshold is arbitrarily chosen, but it reflects a realistic setting where the machines running the applications are billed per unit time. Second, we show the trends only starting from the time instant when all the $K = 30$ repetitions identify at least one feasible point. This allows us to observe the monotonically decreasing trend across all repetitions.

The results confirm those presented in Section VII-B. Specifically, d-MALIBOO is, on average, the best performing algorithm in terms of MAPR decrease across all applications. It achieves the best results on *Query52* and *LiGen*, and ranks second on *Recipe Transcriber* and *Stereomatch*. MALIBOO performs best on *Recipe Transcriber*, but its MAPR is significantly higher on *Stereomatch* and *LiGen*. In the latter case, it struggles to find a single feasible point throughout the iterations, resulting in a delay in the MAPR decrease compared to d-MALIBOO. *OpenTuner* excels on *Stereomatch* but performs

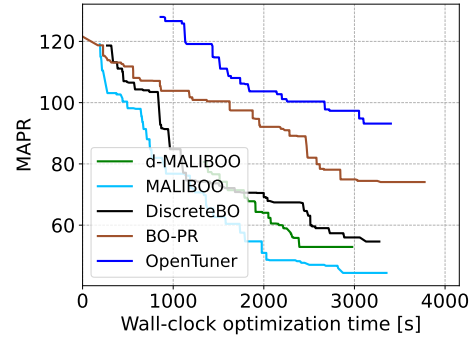


Fig. 7: MAPR trend of all the algorithms on *Recipe Transcriber*, with constraint $T(\mathbf{x}) \leq 300s$ and fixed wall-clock optimization time.

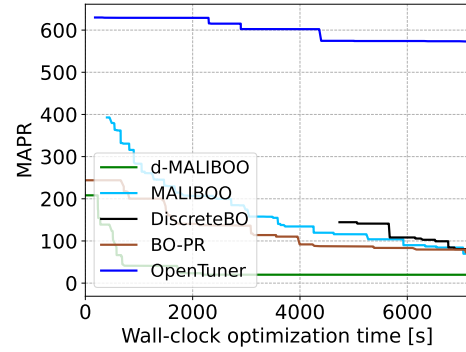


Fig. 8: MAPR trend of all the algorithms on *Query52*, with constraint $T(\mathbf{x}) \leq 405s$ and fixed wall-clock optimization time.

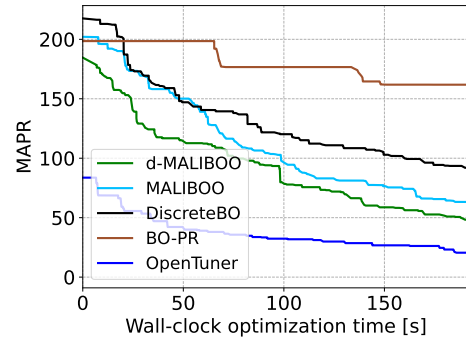


Fig. 9: MAPR trend of all the algorithms on *Stereomatch*, with constraint $T(\mathbf{x}) \leq 17s$ and fixed wall-clock optimization time.

poorly on all other scenarios. Overall, d-MALIBOO exhibits a faster MAPR decrease than the SOTA approaches and remains competitive even when not ranking first.

APPENDIX B
ABLATION STUDY

In this Appendix, we report the results of the SOTA approaches optimizing the *Schwefel* function, as discussed in Section VII-A. Note that results for BO-PR are not included, as the optimization would require several weeks. The numerical results are presented in Table XIII.

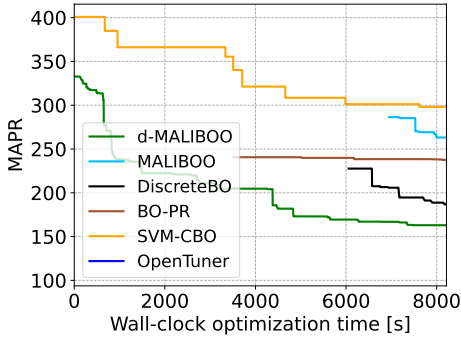


Fig. 10: MAPR trend of all the algorithms on *Stereomatch*, with constraint $R(\mathbf{x}) \leq 2.1$ and fixed wall-clock optimization time.

Algorithm	MAPR[%]	Feas. [%]	Time[s]
d-MALIBOO (reference configuration)	187.47	21.60	499.01
d-MALIBOO (best MAPR of Table III)	178.65	17.71	685.26
d-MALIBOO (best congif. of [28])	190.99	26.65	508.07
MALIBOO	183.64	46.51	495.18
Discrete-BO	179.84	22.17	499.06
SVM-CBO	192.15	11.48	20.07
OpenTuner	232.56	19.73	2.39

TABLE XIII: Results of the ablation study on the SOTA approaches.

The results show that the MAPR of the SOTA approaches is in line with that achieved by the various d-MALIBOO configurations in the ablation study (see Table III). Notably, MALIBOO achieves a higher feasible exploration rate than all other approaches, while the best configuration of d-MALIBOO identified in [28] attains a higher feasible exploration rate than the reference configuration, but also a higher MAPR. This can be attributed to the use of an ML model on the OF. While the Ridge Regression model struggles in scenarios with many disjoint feasible regions, the *probability* approach on the OF (the best configuration in [28]) provides a smoother estimation of the search region, improving the feasible exploration rate. In contrast, the RF model is more effective in reducing the MAPR (configuration achieving the best MAPR in Table III) but attains a lower feasible exploration rate.

These results highlight the difficulty of this particular optimization problem, further emphasizing the value of the robustness and scalability analyses presented in Section VII-A.

APPENDIX C

MANN-WHITNEY TESTS RESULTS

In this Appendix, we report the p-values of the Mann–Whitney tests conducted in the experimental campaign described in Section VII. Recall that the null hypothesis of homogeneity of the MAPRs (i.e., the performance of the approaches are indistinguishable) is rejected when the p-value falls below 5%. Moreover, the * symbol indicates a p-value < 0.01 .

First, Table XIV presents the results of the ablation study of d-MALIBOO on the *Schweffel* function. Next, Tables XV, XVI, XVII, and XVIII report the results of the comparison of the reparameterization techniques of d-MALIBOO on the real-world applications. Finally, Tables XIX, XX, XXI, and XXII

show the Mann–Whitney test results comparing d-MALIBOO with the SOTA approaches on the real-world applications.

ML appr.	ε	ML model	p-value
probability	0.1	Ridge	0.43
indicator	0.01	Ridge	0.33
indicator	0.025	Ridge	0.64
indicator	0.05	Ridge	0.80
indicator	0.2	Ridge	0.49
indicator	0.1	RF	0.07
indicator	0.1	NN	0.55
indicator	0.1	XGBoost	0.78

TABLE XIV: p-values from the Mann-Whitney test comparing the different d-MALIBOO configurations with the reference one.

The tables illustrate the points discussed in more detail in Section VII. Specifically, the differences in MAPR observed in the ablation study are not statistically significant. Moreover, the test confirms the homogeneity of performance across the different reparameterization strategies of d-MALIBOO. Finally, MALIBOO and Discrete-BO, which serve as the foundational approaches for this work, sometimes achieve MAPRs similar to those of d-MALIBOO, particularly on *Recipe Transcriber*. BO-PR shows significant differences in the simplest scenario, with smaller differences in more complex cases. In contrast, SVM-CBO and OpenTuner consistently exhibit different MAPR distributions across all applications.

		$T(\mathbf{x}) \leq 150s$				$T(\mathbf{x}) \leq 300s$				$T(\mathbf{x}) \leq 450s$				$T(\mathbf{x}) \leq 600s$			
#	Config	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
1	Matérn - EI	-	0.10	0.04	0.03	-	0.19	0.26	0.09	-	0.44	0.45	0.51	-	0.48	0.60	0.58
2	Matérn - UCB		-	0.68	0.25		-	0.75	0.48		-	0.24	0.17		-	0.27	0.99
3	RBF - EI			-	0.38			-	0.34			-	0.90			-	0.29
4	RBF - UCB				-				-				-				-

TABLE XV: p-values from the Mann-Whitney test comparing the d-MALIBOO variants on *Recipe Transcriber*.

		$T(\mathbf{x}) \leq 265s$				$T(\mathbf{x}) \leq 335s$				$T(\mathbf{x}) \leq 405s$				$T(\mathbf{x}) \leq 645s$				$T(\mathbf{x}) \leq 2250s$			
#	Config	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
1	Matérn - EI	-	0.86	0.13	0.76	-	0.03	0.18	0.63	-	0.58	0.88	0.71	-	0.13	0.88	0.89	-	0.84	0.79	0.84
2	Matérn - UCB		-	0.16	0.83		-	0.46	0.01		-	0.70	0.95		-	0.13	0.09		-	0.91	0.88
3	RBF - EI			-	0.08			-	0.09			-	0.63			-	0.98			-	0.66
4	RBF - UCB				-				-				-				-				-

TABLE XVI: p-values from the Mann-Whitney test comparing the d-MALIBOO variants on *Query52*.

		$T(\mathbf{x}) \leq 8s$				$T(\mathbf{x}) \leq 10s$				$T(\mathbf{x}) \leq 12s$				$T(\mathbf{x}) \leq 17s$				$T(\mathbf{x}) \leq 20s$				$T(\mathbf{x}) \leq 40s$			
#	Config	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
1	Matérn - EI	-	0.20	0.62	0.24	-	0.36	0.25	0.09	-	0.02	0.07	0.06	-	0.01	0.37	0.01	-	*	0.80	*	-	0.19	0.14	*
2	Matérn - UCB		-	0.81	0.68		-	0.03	0.31		-	*	0.35		-	0.05	0.77		-	*	0.06		-	0.97	0.28
3	RBF - EI			-	0.93			-	*		-	*			-	-	0.06			-	*		-	-	0.30
4	RBF - UCB				-				-			-				-				-				-	

TABLE XVII: p-values from the Mann-Whitney test comparing the d-MALIBOO variants on *Stereomatch*.

		$T(\mathbf{x}) \leq 8s$				$T(\mathbf{x}) \leq 10s$				$T(\mathbf{x}) \leq 12s$				$T(\mathbf{x}) \leq 17s$				$T(\mathbf{x}) \leq 20s$			
#	Config	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
1	Matérn - EI	-	0.21	0.68	0.20	-	0.28	0.28	0.95	-	0.08	0.44	*	-	0.65	0.37	0.06	-	0.44	0.33	0.10
2	Matérn - UCB		-	0.37	0.04		-	0.97	0.20		-	0.33	0.28		-	0.54	0.04		-	0.64	0.25
3	RBF - EI			-	0.11			-	0.20			-	0.08			-	*			-	0.53
4	RBF - UCB				-				-				-				-				-

TABLE XVIII: p-values from the Mann-Whitney test comparing the d-MALIBOO variants on *LiGen*.

	$T(\mathbf{x}) \leq 150s$	$T(\mathbf{x}) \leq 300s$	$T(\mathbf{x}) \leq 450s$	$T(\mathbf{x}) \leq 600s$
MALIBOO	0.87	0.99	0.43	0.11
Discrete-BO	0.97	0.96	0.10	0.66
BO-PR	0.13	0.09	0.16	0.08
SVM-CBO	0.72	*	0.60	N.A.
OpenTuner	*	*	*	*

TABLE XIX: p-values from the Mann-Whitney test comparing d-MALIBOO with the SOTA approaches on *Recipe Transcriber*.

	$T(\mathbf{x}) \leq 265s$	$T(\mathbf{x}) \leq 335s$	$T(\mathbf{x}) \leq 405s$	$T(\mathbf{x}) \leq 645s$	$T(\mathbf{x}) \leq 2250s$
MALIBOO	*	0.31	0.16	0.02	*
Discrete-BO	0.02	0.03	0.46	0.18	0.38
BO-PR	*	*	*	*	*
SVM-CBO	*	0.01	0.57	N.A.	N.A.
OpenTuner	*	*	*	*	*

TABLE XX: p-values from the Mann-Whitney test comparing d-MALIBOO with the SOTA approaches on *Query52*.

	$T(\mathbf{x}) \leq 8s$	$T(\mathbf{x}) \leq 10s$	$T(\mathbf{x}) \leq 12s$	$T(\mathbf{x}) \leq 17s$	$T(\mathbf{x}) \leq 20s$	$T(\mathbf{x}) \leq 40s$
MALIBOO	0.69	0.04	0.13	0.48	0.88	0.29
Discrete-BO	0.79	0.10	0.02	*	0.05	0.31
BO-PR	*	0.50	0.65	1.00	0.06	*
SVM-CBO	*	*	*	*	*	0.02
OpenTuner	0.98	*	*	*	*	*

TABLE XXI: p-values from the Mann-Whitney test comparing d-MALIBOO with the SOTA approaches on *Stereomatch*.

	$R(\mathbf{x}) \leq 2.0$	$R(\mathbf{x}) \leq 2.1$	$R(\mathbf{x}) \leq 2.2$	$R(\mathbf{x}) \leq 2.45$	$R(\mathbf{x}) \leq 2.75$
MALIBOO	0.82	0.25	0.51	0.97	0.04
Discrete-BO	0.17	0.12	0.25	0.09	0.56
BO-PR	*	0.01	0.22	0.94	0.52
SVM-CBO	*	*	*	*	N.A.
OpenTuner	*	*	*	*	*

TABLE XXII: p-values from the Mann-Whitney test comparing d-MALIBOO with the SOTA approaches on *LiGen*.